

Prompt para revisión de código

<https://gemini.google.com/app/625a63b9ba8eb1e4>

User prompt: hazme un promot para verificar un codigo con las siguientes instrucciones

Response: ¡Claro! Aquí tienes un **prompt estructurado y muy efectivo** para pedirle a cualquier modelo de IA (o a mí mismo) que revise y verifique tu código.

Solo copia y pega la plantilla y reemplaza el bloque de código:

Prompt para verificación de código

Actúa como un desarrollador Senior y experto en revisión de código (Code Reviewer).

Por favor, revisa y verifica el siguiente código siguiendo estos puntos minuciosamente:

1. Detección de Errores: Identifica errores sintácticos, de lógica o de ejecución (bugs) que puedan causar fallos o comp
2. Optimización y Rendimiento: Evalúa la eficiencia del código (complejidad temporal y espacial) y sugiere mejoras de re
3. Buenas Prácticas y Legibilidad: Verifica si cumple con las convenciones del lenguaje (naming conventions, estructurac
4. Seguridad: Identifica posibles vulnerabilidades de seguridad (vulnerabilidades comunes, inyección, manejo de datos se
5. Manejo de Excepciones: Evalúa si los casos borde (edge cases) y los errores están bien capturados y gestionados.

Formato de la respuesta:

- Estado del código: (Aprobado / Requiere correcciones / Crítico)
- Lista de problemas encontrados (clasificados por severidad: Alta, Media, Baja).
- Explicación de las mejoras sugeridas.
- Código corregido y optimizado listo para usar.

Aquí está el código a revisar:

```
```[especifica_aquí_el_lenguaje]
// Pega tu código aquí
```

---

```
<ElicitationsGroup message="{Cómo te gustaría proceder?}">
 <Elicitation label="Revisar un código ahora mismo" query="Quiero verificar mi código con este prompt. Aquí está mi cód
 <Elicitation label="Adaptar el prompt para un lenguaje específico" query="Adapta el prompt de revisión de código espec
</ElicitationsGroup>
```

User prompt: *Árboles Binarios de Búsqueda con apoyo de IA* Desarrollar un programa que construya y analice un árbol binario de búsqueda a partir de una oración, utilizando un asistente de IA como herramienta de apoyo durante el desarrollo. El equipo deberá demostrar que puede formular instrucciones claras para la IA, analizar las respuestas obtenidas, verificar el código y corregir posibles errores, en lugar de copiar directamente una solución generada. Una empresa desea analizar las palabras utilizadas en diferentes textos. Para ello, se requiere desarrollar un programa que reciba una oración y almacene sus palabras en un árbol binario de búsqueda. El programa deberá: Solicitar al usuario una línea de texto. Dividir la oración en palabras. Insertar las palabras en un árbol binario de búsqueda. Para realizar la inserción: Si la palabra es alfabéticamente menor que la del nodo actual, deberá insertarse en el subárbol izquierdo. Si es alfabéticamente mayor, deberá insertarse en el subárbol derecho. Si la palabra ya existe, deberá decidirse y documentarse cómo se manejarán los duplicados. Mostrar los recorridos: Inorden Preorden Postorden Calcular y mostrar: Número total de nodos. Número de nodos internos. Palabra con el máximo valor alfabético. Mostrar: Información almacenada en las hojas. Información almacenada en los nodos internos. basate para hacer el promt

Response: Aquí tienes una serie de **prompts estructurados por etapas**, diseñados específicamente para cumplir con el flujo de trabajo guiado, las verificaciones y los casos de prueba solicitados en tu práctica.

## Opción 1: Prompt Maestro (Todo en uno)

Si quieres iniciar pidiéndole a la IA la solución completa bien estructurada:

Actúa como un profesor de estructura de datos y desarrollador senior[cite: 1].

Necesito desarrollar un programa que construya y analice un Árbol Binario de Búsqueda (BST) a partir de una oración ingr

### Requisitos del programa:

1. Entrada: Solicitar al usuario una línea de texto/oración[cite: 1].
2. Procesamiento de texto: Limpiar la oración (eliminar signos de puntuación) y convertir a minúsculas para normalizar l
3. Construcción del Árbol (BST):
  - Dividir la oración en palabras e insertarlas en el árbol[cite: 1].
  - Menores alfabéticamente a la izquierda, mayores a la derecha[cite: 1].
  - Manejo de duplicados: Incluir un contador interno en cada nodo para registrar la frecuencia de palabras repetidas e
4. Recorridos: Mostrar el árbol en Inorden, Preorden y Postorden[cite: 1].
5. Análisis y Métricas:
  - Calcular y mostrar el número total de nodos (palabras únicas)[cite: 1].

- Calcular y mostrar el número de nodos internos[cite: 1].
- Encontrar y mostrar la palabra con el máximo valor alfabético[cite: 1].
- Listar la información/palabras almacenadas en las hojas[cite: 1].
- Listar la información/palabras almacenadas en los nodos internos[cite: 1].

### Por favor proporciona:

1. El código modular y documentado (especifica el lenguaje, ej. Java / Python / C++).
2. Explicación paso a paso de cómo funciona la estructura del nodo y los algoritmos clave (especialmente el conteo de no
3. Propón 3 casos límite adicionales que no sean los típicos (ej. cadenas vacías, solo símbolos, caracteres con acentos)

● **Opción 2: Prompts por Etapas (Recomendado para el Registro de Interacción)**

Para llenar correctamente la **Tabla de registro de interacción con la IA (Etapa 5)**, es muy conveniente hacer las solicitudes por partes de manera gradual:

**Paso 1: Diseño del Nodo y Normalización**

"Actúa como un experto en estructuras de datos. Diseña la clase `Nodo` para un `Árbol Binario de Búsqueda` de palabras en [Lenguaje]. Cada nodo debe guardar la palabra, su frecuencia (para manejar duplicados) y los punteros/referencias a los hijos izquierdo y derecho. Agrega una función inicial para limpiar la oración (quitar puntuación y pasar a minúsculas)."

**Paso 2: Lógica de Inserción y Recorridos**

"Añade al código anterior el método de inserción en el BST siguiendo el orden alfabético. Si la palabra se repite, incrementa el contador de frecuencia. Incluye los métodos para imprimir los recorridos Inorden, Preorden y Postorden."

**Paso 3: Métricas, Hojas y Nodos Internos**

"Agrega funciones recursivas para: 1) Contar el total de nodos, 2) Contar y listar los nodos internos, 3) Mostrar las hojas, y 4) Obtener la palabra con el máximo valor alfabético."

**Paso 4: Solicitud de Casos Límite (Etapa 4)**

"Revisa el código completo generado. Además de los casos comunes (palabras repetidas, mayúsculas, signos de puntuación), propón 3 o 4 casos límite (edge cases) no contemplados y explícame por qué son importantes de probar."

🔧 **Guía de Verificación (Guarda esto para tu reporte)**

Cuando la IA te entregue el código, responde a estas preguntas en tu **Etapa 3** de entrega:

Pregunta de Verificación	Guía para responder en tu reporte
¿Qué problema resuelve este código?	Almacena y analiza la frecuencia y estructura alfabética de palabras en un texto mediante un BST.
¿Cómo funciona?	Normaliza la cadena, construye el árbol comparando cadenas lexicográficamente y recorre recursivamente los nodos para extraer métricas.
¿Qué entradas necesita?	Una línea de texto/oración del usuario.
¿Qué resultado produce?	Los 3 recorridos, conteo de nodos totales/internos, hoja/internos impresos y palabra alfabéticamente mayor.
¿Qué modificaciones realizamos y por qué?	(Ejemplo) "Ajustamos el manejo de acentos/tildes en la limpieza de datos porque la IA no lo incluía por defecto."